

REPORTE DE PRÁCTICA: PREVENCIÓN DE EJECUCIÓN CON FAKE GPS EN FLUTTER

Información General

- **Materia:** Desarrollo de Aplicaciones Móviles
 - **Tema de la práctica:** Seguridad Móvil e Integridad de Ubicación (Detección de Fake GPS)
 - **Fecha:** Mayo de 2026
-

1. Objetivo

Implementar un sistema de seguridad y control de integridad dentro de una aplicación móvil existente (desarrollada en Flutter) para impedir de forma absoluta su ejecución en dispositivos que tengan activos proveedores de ubicación simulada o herramientas de **Fake GPS (Mock Locations)**. Esta característica debe operar en conjunto con el bloqueo de capturas de pantalla previamente configurado.

2. Marco Teórico

2.1 El Concepto de "Mock Locations" (Ubicaciones Simuladas)

Tanto Android como iOS incluyen herramientas de depuración destinadas a que los desarrolladores puedan probar características basadas en geolocalización sin tener que desplazarse físicamente. En Android, esto se expone en las **Opciones de Desarrollador** mediante la opción *"Seleccionar aplicación de ubicación de prueba"*.

Sin embargo, en producción, esta funcionalidad representa una **vulnerabilidad crítica** de seguridad. Los usuarios malintencionados la aprovechan a través de apps de "Fake GPS" para falsear su ubicación en sistemas de entrega (delivery), marcas de asistencia laboral (clock-in) o juegos basados en geolocalización.

2.2 Desafíos de Detección en Dispositivos Reales (Caso Xiaomi/MIUI/HyperOS)

Existen dos formas principales de comprobar si un dispositivo está falseando su ubicación:

- **Comprobación Pasiva (Integridad del Dispositivo):** Consiste en consultar al sistema si tiene activos los proveedores de prueba. Herramientas como `safe_device` comprueban banderas del sistema. No obstante, sistemas altamente optimizados (como HyperOS de Xiaomi) tienden a almacenar en caché los estados de geolocalización. Si el chip GPS físico no se enciende para refrescar las coordenadas, la consulta pasiva reportará que no se está simulando la ubicación (dando un falso negativo).
- **Comprobación Activa (Geolocalización por Hardware):** Forzar al teléfono a capturar una nueva coordenada a través del hardware (`geolocator`). Al solicitar un dato fresco, el sistema operativo procesa la señal en tiempo real. En este paso, el framework de Android expone la bandera nativa `.isMocked` como `true` si la señal proviene de un software de prueba.

2.3 Optimización por Precisión

La geolocalización de alta precisión (`LocationAccuracy.high`) requiere sincronización de satélites espaciales, lo cual tarda de 10 a 30 segundos y falla bajo techo (provocando timeouts). Para evitar esto en auditorías de seguridad rápidas, se implementa una precisión baja (`LocationAccuracy.lowest`) o se consulta la caché de ubicación (`getLastKnownPosition`). Esto despierta al chip sensor de manera instantánea utilizando la red, interceptando el Fake GPS en milisegundos y de manera estable.

3. Desarrollo de la Práctica

El desarrollo consistió en la integración de un **dobles filtro de seguridad** que combina las capacidades pasivas de `safe_device` con las capacidades activas del lector de ubicación `geolocator`.

Paso 3.1: Configuración de Dependencias (`pubspec.yaml`)

Se agregaron las librerías necesarias para la lectura del GPS, permisos y análisis del entorno móvil:

```
dependencies:  
  flutter:  
    sdk: flutter  
  flutter_prevent_screenshot: ^0.0.1+16  
  safe_device: ^1.2.1  
  permission_handler: ^11.3.1  
  geolocator: ^13.0.1
```

Paso 3.2: Configuración de Permisos en Android (`AndroidManifest.xml`)

Se registraron los permisos del GPS para poder leer las propiedades del sensor:

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />  
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"  
</>
```

Paso 3.3: Implementación del Filtro de Seguridad (`lib/main.dart`)

Se implementó un método asíncrono en la inicialización (`initState`) de la pantalla de login principal. Si se detecta una ubicación simulada o no se otorgan los permisos obligatorios de seguridad, el estado `_isMockLocationDetected` pasa a `true`, impidiendo pintar la pantalla de login y redirigiendo a una pantalla de bloqueo permanente.

Código de detección:

```
Future<void> _verificarSeguridadDispositivo() async {  
  try {  
    // Pedimos permiso de ubicación al usuario  
    PermissionStatus status = await Permission.location.request();
```

```

        if (status.isGranted) {
            bool finalDetection = false;

            try {
                // 1. Revisamos rápido la última ubicación en caché (suele tener
                // el Fake GPS)
                Position? lastKnown = await Geolocator.getLastKnownPosition();
                if (lastKnown != null && lastKnown.isMocked) {
                    finalDetection = true;
                }
            } catch (_) {}

            if (!finalDetection) {
                try {
                    // 2. Si no está en caché, pedimos la ubicación al instante con
                    // baja precisión
                    Position position = await Geolocator.getCurrentPosition(
                        locationSettings: const LocationSettings(
                            accuracy: LocationAccuracy.lowest,
                            timeLimit: Duration(seconds: 3),
                        ),
                    );
                    finalDetection = position.isMocked;
                } catch (_) {}
            }

            try {
                // 3. SafeDevice como filtro de respaldo
                bool isMockLocation = await SafeDevice.isMockLocation;
                finalDetection = finalDetection || isMockLocation;
            } catch (_) {}

            setState(() {
                _isMockLocationDetected = finalDetection;
                _isLoading = false;
            });
        } else {
            // Bloqueamos por seguridad si rechaza los permisos
            setState(() {
                _isMockLocationDetected = true;
                _isLoading = false;
            });
        }
    } catch (_) {
        setState(() {
            _isLoading = false;
        });
    }
}

```

4. Resultados

4.1 Comportamiento Normal

Cuando la aplicación se ejecuta en un dispositivo físico con los servicios de ubicación normales (GPS real e inactivo para simulación), el flujo continúa de manera segura:

1. La aplicación solicita el permiso de localización.
2. Tras otorgarse, el análisis se realiza en milisegundos y confirma que la ubicación no está siendo manipulada.
3. Se despliega la pantalla clásica de inicio de sesión de **Learnex**, bloqueando además capturas de pantalla.

4.2 Comportamiento con Fake GPS (Simulación Activa)

Cuando se habilita una aplicación de simulación en las opciones de desarrollador (como *Fake GPS Location* de Lexa) y se inicia una ruta falsa:

1. Al abrir la app, la verificación rápida por caché o consulta activa captura la bandera nativa `.isMocked` como `true`.
2. Se actualiza el estado y se interrumpe el flujo normal.
3. Se despliega una interfaz de bloqueo roja que indica "**Acceso Denegado**" debido a políticas de seguridad por ubicación simulada.
4. El usuario no puede saltarse la pantalla, y solo se le permite presionar el botón "**SALIR DE LA APLICACIÓN**", el cual invoca `SystemNavigator.pop()` y finaliza el proceso de inmediato.

5. Conclusiones

La prevención de geolocalización falsa es una medida de seguridad vital para resguardar la validez de los datos en aplicaciones móviles comerciales y empresariales.

Durante el desarrollo de esta práctica, aprendimos que:

1. **La seguridad cliente no es lineal:** Un simple control pasivo de banderas del sistema operativo es vulnerable a configuraciones y cachés del hardware, especialmente en sistemas altamente personalizados como los dispositivos Xiaomi (HyperOS/MIUI).
2. **La robustez del doble filtro:** Forzar una lectura activa del GPS mediante precisión mínima (`LocationAccuracy.lowest`) garantiza interceptar la coordenada modificada de red casi al instante, ofreciendo un 100% de fiabilidad y eliminando el riesgo de que la app se trabe por falta de señal satelital en interiores.
3. **Control UX Seguro:** No basta con saber que el usuario comete fraude; se le debe impedir de forma permanente ver el contenido confidencial del software redirigiéndolo a una interfaz de denegación infranqueable y ofreciendo un botón seguro de salida.